S1. Introdução a DevOps

M13. DevOps

ServiceNow
Faculdade de Ciências da Universidade de Lisboa
UPskill – Programa Digital SKILLS & JOBS (2025/26)
Mário Calha

Content

- Core concepts
- CI/CD
- IaC
- Monitoring and observability
- DevOps
- Security (DevSecOps)
- GitHub Actions

What is DevOps?

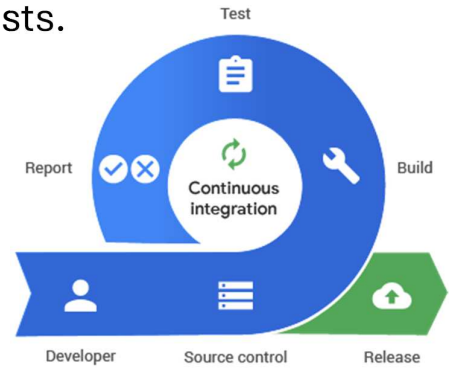
- DevOps is a cultural and technical approach that unifies software development (Dev) and IT operations (Ops) teams to deliver software faster, with higher quality, and more reliably.
- Key Principles:
  - **Collaboration**: Breaking down silos between development and operations teams
  - **Automation**: Automating repetitive tasks throughout the software delivery lifecycle
  - **Continuous Improvement**: Constantly measuring, learning, and optimizing processes
  - **Customer Focus**: Delivering value to end users quickly and reliably

DevOps Lifecycle

- The DevOps lifecycle consists of eight interconnected phases:
  1. **Plan** - Define business goals, gather requirements, and plan sprints
  2. **Code** - Write, review, and manage source code using version control
  3. **Build** - Compile code, run automated tests, and create artifacts
  4. **Test** - Perform automated testing including unit, integration, and security tests
  5. **Release** - Manage deployments and ensure consistency across environments
  6. **Deploy** - Push changes to production using automated deployment pipelines
  7. **Operate** - Monitor infrastructure, manage resources, and handle scaling
  8. **Monitor** - Track performance metrics, errors, and user feedback

Continuous Integration (CI)

- Developers merge code changes into a shared repository multiple times per day, triggering automated builds and tests.
- Benefits:
  - Early detection of bugs and integration issues
  - Reduced risk through smaller, incremental changes
  - Faster feedback loop for developers
  - Improved code quality through automated testing
- Best Practices:
  - Commit code frequently (at least once per day)
  - Maintain a fast, reliable build pipeline (under 10 minutes)
  - Keep the build server always green
  - Run tests in isolation to ensure reliability

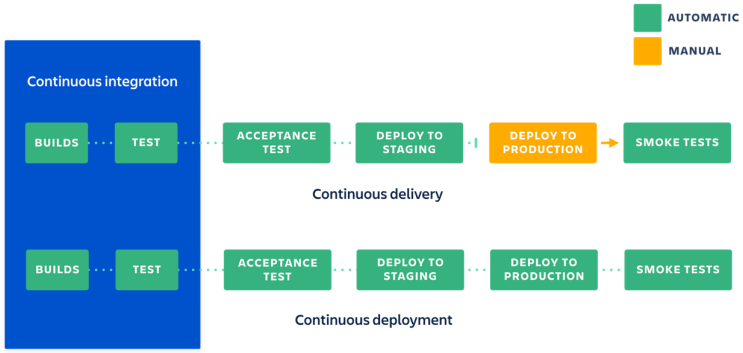


Continuous Delivery (CD)

- Code changes are automatically prepared for release to production after passing all automated tests.
- Key Features:
  - Automated deployment pipelines
  - Environment consistency across dev, staging, and production
  - One-click or automated rollbacks
  - Database migration management
- Goals:
  - Make deployments routine, low-risk events
  - Reduce time from development to production
  - Increase deployment frequency without increasing risk

Continuous Deployment vs. Continuous Delivery

Aspect	**Continuous Delivery**	**Continuous Deployment**
Automation Level	Automated to staging, manual to production	Fully automated end-to-end
Production Release	Requires manual approval	Happens automatically
Risk	Lower - human gatekeeper	Higher - requires robust testing
Use Cases	Most enterprise applications	SaaS, startups with mature testing



Infrastructure as Code (IaC)

- Managing and provisioning infrastructure through code and version control, rather than manual processes.
- Benefits:
  - **Consistency**: Identical environments every time
  - **Version Control**: Track changes, rollback when needed
  - **Replicability**: Spin up identical environments quickly
  - **Documentation**: Code serves as documentation
- Popular Tools:
  - Terraform (cloud-agnostic)
  - AWS CloudFormation
  - Ansible (configuration management)
  - Pulumi (infrastructure as actual code)

Microservices Architecture

- An architectural style that structures an application as a collection of loosely coupled, independently deployable services.
- Characteristics:
  - Each service owns its own database
  - Services communicate via APIs
  - Services can be developed and deployed independently
  - Polyglot programming (different services use different languages)
- DevOps Implications:
  - Independent deployment pipelines per service
  - Service-specific monitoring and logging
  - Distributed tracing requirements
  - Contract testing between services

Containerization & Docker

- Containers are lightweight, standalone executable packages that include everything needed to run software.
- Docker Benefits:
  - Consistency: Same environment from dev to production
  - Isolation: Services run independently without conflicts
  - Portability: Run anywhere Docker is installed
  - Efficiency: Share OS kernel, lighter than VMs
- Key Concepts:
  - Dockerfile: Blueprint for building container images
  - Image: Read-only template for containers
  - Container: Running instance of an image
  - Registry: Storage and distribution for images

Kubernetes

- Kubernetes is an open-source container orchestration platform that automates deployment, scaling, and management of containerized applications.
- Core Features:
  - **Self-healing**: Automatically restarts failed containers
  - **Horizontal scaling**: Scale applications up and down automatically
  - **Load balancing**: Distributes traffic across containers
  - **Rolling updates**: Deploy new versions without downtime
- Key Concepts:
  - **Pods**: Smallest deployable units
  - **Deployments**: Manage replica sets of pods
  - **Services**: Stable network endpoint for pods
  - **Namespaces**: Virtual clusters within a physical cluster

Monitoring & Observability

- The Three Pillars:
  1. **Metrics**: Quantitative measurements over time (CPU usage, request rates, error rates)
  2. **Logs**: Timestamped records of events (application logs, system logs)
  3. **Traces**: End-to-end request paths across distributed systems
- Tools:
  - Prometheus + Grafana (metrics and visualization)
  - ELK Stack (Elasticsearch, Logstash, Kibana)
  - Jaeger or Zipkin (distributed tracing)
  - Datadog, New Relic, or CloudWatch (comprehensive APM – Application Performance Monitoring)

DevOps Culture & Team Practices

- Cultural Pillars:
  - **Shared Ownership**: Both dev and ops share responsibility for production systems
  - **Psychological Safety**: Teams feel safe to experiment and fail
  - **Blameless Postmortems**: Focus on systems, not individuals
  - **Transparency**: Open communication across all teams
- Team Structures:
  - **Platform Teams**: Build internal platforms for product teams
  - **Embedded SRE**: Operations expertise embedded in development teams
  - **Jellyfish Model**: Mix of centralized and decentralized expertise

DevOps Metrics & KPIs

- DORA Metrics (DevOps Research and Assessment):
  1. **Deployment Frequency**: How often code is deployed to production
  2. **Lead Time for Changes**: Time from commit to production deployment
  3. **Mean Time to Recovery (MTTR)**: How quickly service is restored after failure
  4. **Change Failure Rate**: Percentage of deployments causing failures
- Organizational Metrics:
  - Availability percentage (target: 99.9%+)
  - Infrastructure cost per transaction
  - Automated test coverage percentage
  - Mean time between failures (MTBF)

Security in DevOps (DevSecOps)

- Integrating security practices throughout the DevOps lifecycle rather than treating it as an afterthought.
- Security Practices:
  - Shift Left: Security testing early in the development process
  - Infrastructure Security: Secure by design principles
  - Secrets Management: Secure storage for credentials (HashiCorp Vault, AWS Secrets Manager)
  - Compliance as Code: Automated compliance checks
- Tools:
  - SAST (Static Application Security Testing)
  - DAST (Dynamic Application Security Testing)
  - Container scanning (Trivy, Clair)
  - Dependency vulnerability scanning (Snyk, Dependabot)

Popular DevOps Tools Ecosystem

- **Version Control**: Git, GitHub, GitLab, Bitbucket
- **CI/CD**: Jenkins, GitLab CI, GitHub Actions, CircleCI, ArgoCD
- **Container**: Docker, Podman, containerd
- **Orchestration**: Kubernetes, Docker Swarm
- **IaC**: Terraform, Pulumi, CloudFormation, Ansible
- **Monitoring**: Prometheus, Grafana, Datadog, New Relic
- **Collaboration**: Slack, Jira, Confluence, PagerDuty

Getting Started with DevOps

- Implementation Roadmap:
  1. Assess Current State
     - Map existing processes and identify bottlenecks
     - Identify team skills and tool gaps
     - Set clear, measurable goals
  2. Start Small
     - Choose one application or team for pilot
     - Implement version control if not already in place
     - Add basic CI/CD pipeline
  3. Automate Gradually
     - Automate testing first
     - Add automated deployments
     - Implement infrastructure as code
  4. Measure & Improve
     - Track DORA metrics
     - Conduct blameless retrospectives
     - Continuously iterate and refine

Summary & Key Takeaways

- Core DevOps Messages:
  - DevOps bridges the gap between development and operations teams
  - Automation is the foundation: build, test, deploy, and infrastructure
  - Continuous improvement through measurement and feedback
  - Security must be integrated throughout the pipeline (DevSecOps)
  - Culture matters as much as tools and processes
  - Start small, measure results, and iterate continuously

- **Remember**: DevOps is not a destination but a journey of continuous improvement toward delivering better software faster.

GitHub Actions

- **Actions**
  - end-to-end GitHub-centric Software Development Life Cycle process
  - provides an automated platform and framework
  - **Automated platform**
    - a way to create and execute automated workflows tied to GitHub events
  - **Framework**
    - is composed of a core set of related components in GitHub. These components can be put together to execute simple or complex automation in an understandable and predictable way. And this automation is stored in the repository as code.
- Actions Marketplace
  - public registry where actions can be published and shared

  https://github.com/marketplace?type=actions

CI/CD today

- When CI/CD first came on the scene, dedicated tools such as Jenkins were the primary means of creating pipelines.
  - These tools were flexible—arguably, too flexible.
  - You had to work hard to tie together individual parts into a pipeline.
- Today's CI/CD pipelines can be very complex and can include
  - advanced testing,
  - multiple integration levels, and
  - automated deployments/releases.
- GitHub Actions allows you to create workflows as complex as needed to handle these types of operations without ever leaving GitHub's ecosystem.

GitHub pipeline

- Workflow (pipeline)
  - Chain together smaller units of work called jobs.
  - Some change or event causes a series of automated actions to happen automatically in response.
  - Set of code that defines a sequence and set of steps to execute, similar to a script or a program.
  - The file itself must be coded in YAML format and stored in the `<repository>/.github/workflows` directory.
- Job
  - Jobs are what you often might see in other applications as stages, meaning parts of a larger process that perform a distinct and separate function.

Starter workflows

- Main categories
  - Deployment
    - A set of example workflows for creating deployable objects (like containers) and then deploying them to various cloud platforms
  - Security
    - Primarily a set of code-scanning workflows using various security platforms and their tools
  - Continuous Integration
    - A large number of workflows that cover the areas of building, testing, and/or publishing for a large number of different programming languages and tools
  - Automation
    - Some simple examples for basic automation, including a hello world type, one that demonstrates how to trigger a workflow manually, and a couple that deal with other GitHub constructs such as pull requests and issues
  - Pages
    - Workflows to package/deploy sites using common tools like Gatsby, Astro, Jekyll, etc.

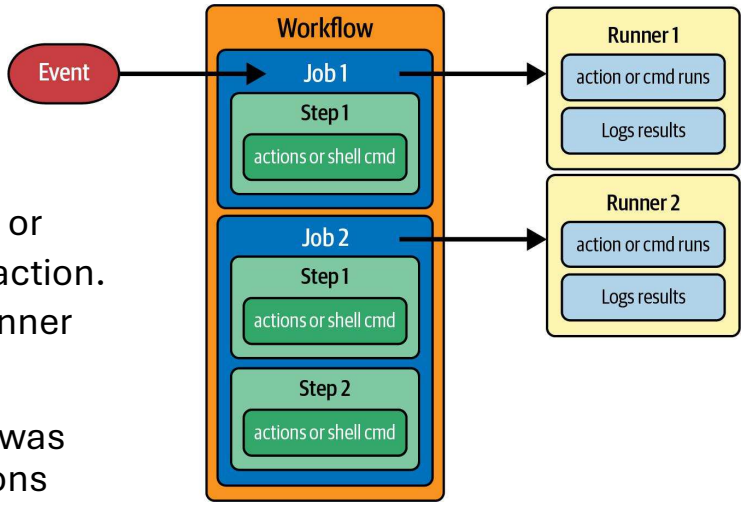
https://github.com/actions/starter-workflows

Costs involved

- Free model
  - GitHub Actions is free if either or both of the following two conditions are true:
    - The repositories you use with actions are public.
    - The systems you execute the actions on (the runners) are your own (rather than using the ones provided by GitHub).
- Paid model
  - There are two types of items you pay for with GitHub Actions:
    - Storage: Actions allow you to store artifacts and packages on GitHub's resources. After a certain point, the amount of storage you're using for artifacts and packages will start to cost you.
    - Minutes: Actions require processing time on virtual systems.

GitHub Actions flow

- Jobs
  - run in parallel (by default)
  - are made of steps
- Step
  - either runs a Shell command, or
  - invokes a predefined GitHub action.
  - All steps are executed on a runner
- Runner
  - is a server or a container that was setup to support GitHub Actions



Triggering workflows

- Events trigger workflows.
  - They serve as the signal for work to start happening if a GitHub Actions workflow is present and the triggering event matches the start conditions specified in the workflow.
- An event can be defined in several different ways:
  - A person or a process does some **operation in a GitHub repository**.
  - A matching **external trigger** happens—that is, an event from outside of GitHub.
  - A **schedule** is set up to run a workflow at particular times or intervals.
  - A workflow is initiated **manually**, without an operation having to be done first.

Triggering workflows

- The workflow can respond to a single event such as when a push happens:
```
on: push
```
- The workflow can respond to a list (multiple events):
```
on: [push, pull_request]
```
- The workflow can respond to event types with qualifiers, such as **branches**, **tags**, or file **paths**:
```
on:
  push:
    branches:
      - main
      - 'rel/v*'
    tags:
      - v1.*
      - beta
    paths:
      - '**.ts'
```

Triggering workflows

- The workflow can execute on a specific schedule or interval (using standard cron syntax):

```
on:
  scheduled:
    - cron: '30 5,15 * * *'
```

- The workflow can respond to specific manual events:

```
on: [workflow-dispatch, repository-dispatch]
```

- The workflow can be called from other workflows (referred to as a reuse event):

```
on: workflow_call
```

- The workflow can respond to common activities on GitHub items, such as adding a comment to a GitHub issue:

```
on: issue_comment
```


Components

- Steps
  - They consist of either invocations of a predefined action or a shell command to be run on the runner. Any shell commands are executed via a **run clause**. And any predefined actions are pulled in via a **uses clause**.
  - The steps keyword indicates the start of a series of steps to be run sequentially.
- Note that steps can have a name associated with them as well:

```
steps:
- uses: actions/checkout@v3
- name: setup Go version
  uses: actions/setup-go@v2
  with:
    go-version: '1.14.0'
- run: go run helloworld.go
```

Components

- Runners
  - Runners are the physical or virtual computers or containers where the code for a workflow is executed. They can be systems provided and hosted by GitHub (and run within their control), or they can be instances you set up, host, and control.
- In a workflow file, runners are defined for jobs simply via the runs-on clause.

```
runs-on: ubuntu-latest
```


Components

- Jobs
  - Jobs aggregate steps and define which runner to execute them on. An individual job is usually targeted towards accomplishing one particular goal of the overall workflow.
  - An example could be a workflow that implements a CI/CD pipeline with separate jobs for building, testing, and packaging.
  - A job in a workflow is like a function or procedure in a programming language.
  - Success or failure is displayed at the level of the job, not the individual steps. If you need more granular reports of success or failure displayed at the top level, you may want to put fewer steps in a job. Or, if you need less-granular indications of whether a set of steps succeeded or failed, you might put more steps into a job.

Components

- Building on the steps and runners previously shown, the next listing shows a simple job that does the checkout and setup and performs a build:

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: setup Go version'
        uses: actions/setup-go@v2
        with:
          go-version: '1.14.0'
      - run: go run helloworld.go
```


Components

- Workflow
  - A workflow is like a pipeline. At a high level, it first defines the types of inputs (events) that it will respond to and under what conditions it will respond to them.
  - The response, if the events and conditions match, is to then execute the series of jobs in the workflow, which, in turn, execute the steps for each job.
  - The overall flow is like a continuous integration process in that it responds to a particular kind of change and kicks off an automated sequence of work.

Components

- A simple workflow for processing Go programs built on the previous definitions:

```
name: Simple Go Build

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
step 1  - uses: actions/checkout@v3
step 2  - name: Setup Go version
          uses: actions/setup-go@v2
          with:
            go-version: '1.15.1'
step 3  - run: go run hello-world.go
```

This workflow is triggered when a push operation is done to the branch main in this GitHub repository.

This job will be executed on a runner system, hosted by GitHub, provisioned with a standard ubuntu operating system image.

The first step is done via pulling in a predefined action. Note the way this is referenced. actions/checkout@v3 refers to the relative path after github.com, so this really says it is going to run/use the action defined at github.com/actions/checkout.

The setup-go action needs a parameter—the version of Go to use. The parameter is passed as an input to the action via a with clause.

The command is to execute the go run command on an example file in the repository.

Components

- In order to be found, matched to event conditions, and executed automatically, this code needs to be stored in a YAML file in a special directory in a GitHub repository:

  `.github/workflows`

- The file extension here needs to be either `.yml` or `.yaml`, denoting YAML structure and syntax.
- If you push the `.github/workflows/simple-go-build.yml` file and a corresponding `helloworld.go` file to a GitHub repository, you can see the workflow actually run right away.

Summary

- Workflows are like software pipelines. They can be initiated when a triggering event occurs (like continuous integration), and they aggregate one or more jobs to accomplish their overall task. Each job in turn aggregates one or more steps to do smaller units of work. The execution of the steps in a single job results in a success/failure outcome for the job, which feeds back into success/failure for the overall workflow.
- Each job declares what kind of runner system (operating system and version) it will run in. And, at the lowest level, steps can invoke predefined GitHub Actions or run simple commands on that system.